



Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS341B: Compiler Design

Prakash C O, Associate Professor, Department of CSE

Abstract Syntax Tree (AST) or Syntax Tree (ST)

- The AST is a high-level intermediate representation. Because of its rough correspondence to a parse tree, the parser can build an AST directly.
- An **abstract syntax tree (AST)**, or just **syntax tree**, is a tree representation of the abstract syntactic structure of source code written in a programming language. Each node of the tree denotes a construct occurring in the source code.
- **The syntax is "abstract" in the sense that it does not represent every detail appearing in the real syntax, but rather just the structural, content-related details.** For instance, grouping parentheses are implicit in the tree structure, and a syntactic construct like an if-condition-then expression may be denoted by means of a single node with three branches.
- This distinguishes abstract syntax trees from concrete syntax trees, traditionally designated parse trees, which are typically built by a parser during the source code translation and compiling process. Once built, additional information is added to the AST by means of subsequent processing, e.g., contextual analysis.

Application in compiler

- Abstract syntax trees are data structures widely used in compilers to represent the structure of program code.
- **An AST is usually the result of the syntax analysis phase of a compiler.**
- An AST often serves as an intermediate representation of the program through several stages that the compiler requires, and has a strong impact on the final output of the compiler.

Motivation

An AST has several properties that aid the further steps of the compilation process:

- An AST can be edited and enhanced with information such as properties and annotations for every element it contains. Such editing and annotation is impossible with the source code of a program, since it would imply changing it.
- Compared to the source code, an AST does not include inessential punctuation and delimiters (braces, semicolons, parentheses, etc.).

- An AST usually contains extra information about the program, due to the consecutive stages of analysis by the compiler. For example, it may store the position of each element in the source code, allowing the compiler to print useful error messages.

Design

The design of an AST is often closely linked with the design of a compiler and its expected features. Core requirements include the following:

- Variable types must be preserved, as well as the location of each declaration in source code.
- The order of executable statements must be explicitly represented and well defined.
- Left and right components of binary operations must be stored and correctly identified.
- Identifiers and their assigned values must be stored for assignment statements.

These requirements can be used to design the data structure for the AST.

Some operations will always require two elements, such as the two terms for addition. However, some language constructs require an arbitrarily large number of children, such as argument lists passed to programs from the command shell. As a result, an AST used to represent code written in such a language has to also be flexible enough to allow for quick addition of an unknown quantity of children.

Another major design requirement for an AST is that it should be possible to unparse an AST into source code form. The source code produced should be sufficiently similar to the original in appearance and identical in execution, upon recompilation.

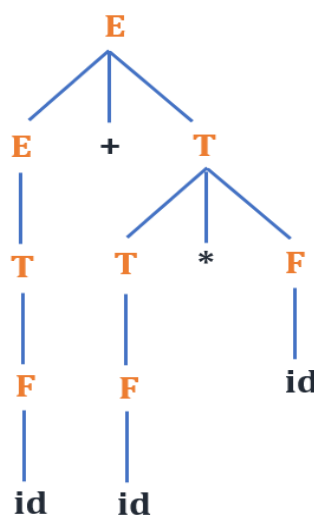
Usage

- The AST is used intensively during semantic analysis, where the compiler checks for correct usage of the elements of the program and the language.
- The compiler also generates symbol tables based on the AST during semantic analysis. A complete traversal of the tree allows verification of the correctness of the program.
- After verifying correctness, the AST serves as the base for code generation. The AST is often used to generate an intermediate representation (IR), sometimes called an intermediate language, for the code generation.

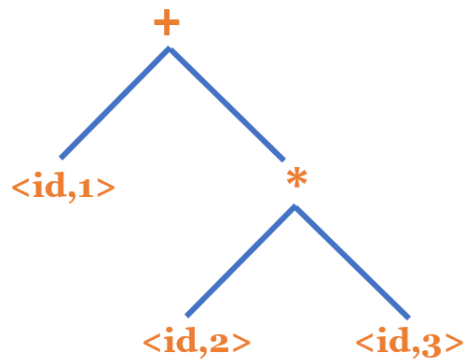
Example 1: Arithmetic Expression $a+b*c$ (The token stream $\langle id,1 \rangle \langle + \rangle \langle id,2 \rangle \langle * \rangle \langle id,3 \rangle$)

A parse tree is a labelled tree representation of a derivation.

- The interior nodes are nonterminals.
- The leaf nodes are terminals

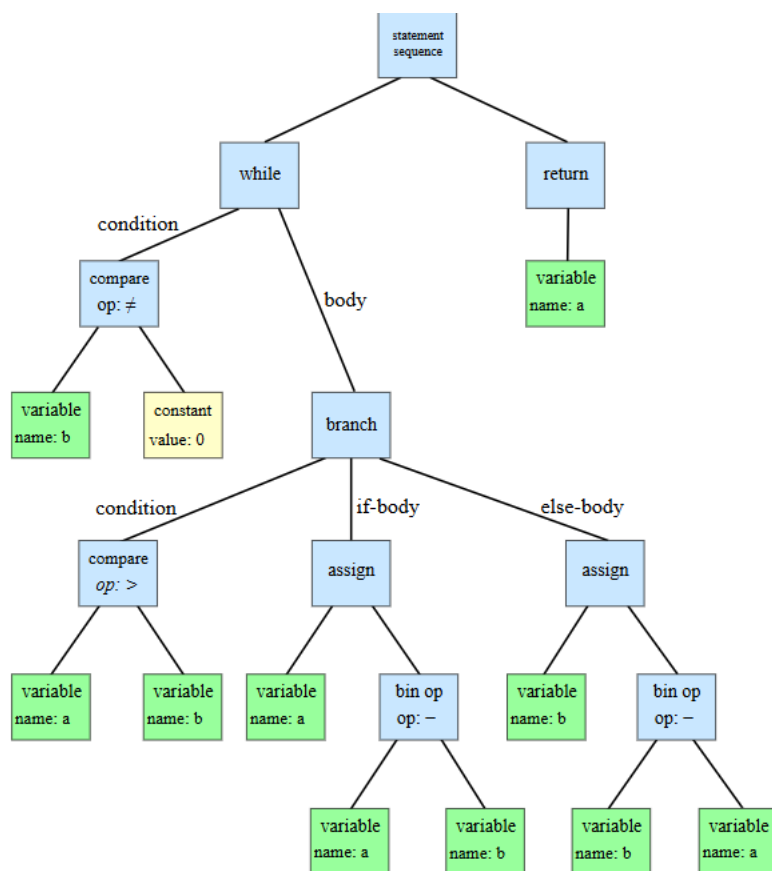


The corresponding syntax tree

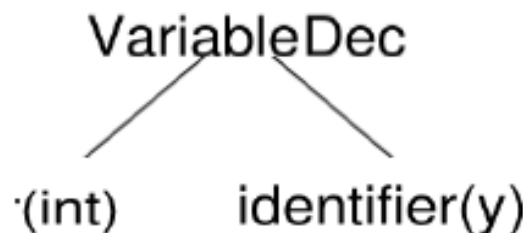
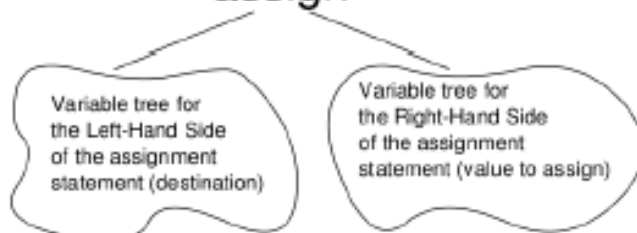
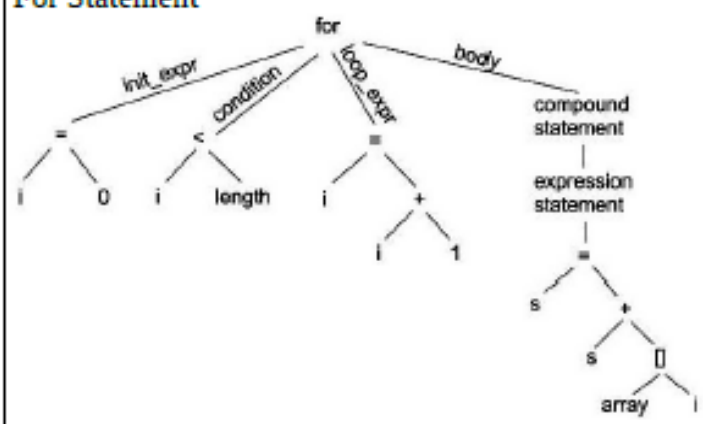


Example 2: An abstract syntax tree for the following code for the Euclidean algorithm

```
while b ≠ 0:  
  if a > b:  
    a := a - b  
  else:  
    b := b - a  
return a
```



Constructing Syntax tree for Different statements

Decl -> Type VarList Type -> int float char VarList -> VarList, id id	AST – Variable Declaration int y;  <pre> graph TD VariableDec --> int["(int)"] VariableDec --> identifier["identifier(y)"] </pre>
AssignExpr -> id = E E -> E + T E -> T T -> T * F T -> F F -> num id (E)	AST – assignment Statements assign 
Sample CFG S -> if(cond){S} if(cond){S}else{S} while(cond){S} for(AssignExpr cond AssignExpr){S} AssignExpr S; S epsilon UnaryExpr Decl break ; continue ; return <expr> ; goto <id> ; epsilon cond -> expr expr logOp expr expr -> relexp logexp E relexp -> E relOp E logexp -> E logOp E logOp -> && relOp -> < > <= >= != == AssignExpr -> id = E; E -> E + T T T -> T * F F F -> id num (E) UnaryExpr -> ++ UE UE++ --UE UE-- UE -> id (E) Decl -> Type VarList; Type -> int float char VarList -> VarList, id id	If Statement  While statement  For Statement 

Difference between Abstract Syntax tree and Parse tree

Parse Tree (Concrete Syntax Tree)	Abstract Syntax Tree
- Concrete syntax: what the programmer wrote - tree representation of grammar derivation	- Abstract syntax: what the compiler needs - condensed form of parse tree. - Abstract tree has less information than the concrete tree. - Operators and keywords do not appear as leaves - Chains of single productions are collapsed
<pre> graph TD S --> IF S --> B S --> THEN S --> S1 S --> ELSE S --> S2 </pre>	<pre> graph TD If-then-else --> B If-then-else --> S1 If-then-else --> S2 </pre>
$E \rightarrow E + T$ $E \rightarrow T$ $T \rightarrow T * F$ $T \rightarrow F$ $F \rightarrow \text{num} \mid \text{id} \mid (E)$ <pre> graph TD E1[E] --> E2[E] E1 --> P1[+] E1 --> T1[T] E2 --> T2[T] T2 --> F1[F] F1 --> 3 T1 --> T3[T] T1 --> M1[*] T1 --> F2[F] T3 --> F3[F] F3 --> 4 F2 --> 5 </pre> Parse tree for $3 + 4 * 5$ $3 * (4 + 5)$	<pre> graph TD Plus1[+] --> 3 Plus1 --> Star1[*] Star1 --> 4 Star1 --> 5 </pre> Abstract Syntax Tree for $3 + 4 * 5$ What about parentheses? Do we need to store them? - Parenthesis information is store in the shape of the tree - No extra information is necessary <pre> graph TD Star2[*] --> Plus2a[+] Star2 --> Plus2b[+] Plus2a --> IL3a[Integer_Literal(3)] Plus2a --> IL4a[Integer_Literal(4)] Plus2b --> IL5b[Integer_Literal(5)] Plus2b --> IL6b[Integer_Literal(6)] </pre> $(3 + 4) * (5 + 6)$
$((4))$	<pre> graph TD Star3[*] --> Plus3a[+] Star3 --> Plus3b[+] Plus3a --> IL3c[Integer_Literal(3)] Plus3a --> IL4c[Integer_Literal(4)] Plus3b --> IL5d[Integer_Literal(5)] Plus3b --> IL6d[Integer_Literal(6)] </pre> Integer_Literal(4)

References:

- https://en.wikipedia.org/wiki/Abstract_syntax_tree
 -
-

Parse Tree

- Also called a **concrete syntax tree**.
- Represents the **complete grammatical structure** of the source code according to the grammar.
- Includes **all details** from the grammar, such as non-terminal and terminal symbols.
- Often **very detailed and large**, because it reflects every production rule used.

Syntax Tree (Abstract Syntax Tree – AST)

- A **simplified version** of the parse tree.
- Focuses on the **essential structure** of the program, omitting unnecessary grammar details (like parentheses or certain keywords).
- Used by compilers for **semantic analysis**, **optimization**, and **code generation**.

In practice:

- **The parser usually first produces a parse tree, and then converts it into an AST for further processing.**

A Parse Tree is called a Concrete Syntax Tree because:

- It represents the **exact structure of the source code** as dictated by the grammar rules.
- Every detail from the grammar is preserved:
 - All **non-terminal symbols** (like <expression>, <term>, <factor>).
 - All **terminal symbols** (actual tokens like +, *, a, b, c).
- It mirrors the **concrete syntax** of the language, which includes all syntactic elements—even those that are not semantically important (e.g., parentheses, keywords).

In contrast, an **Abstract Syntax Tree (AST)** omits unnecessary details and focuses on the **logical structure** of the program.

A Syntax Tree is called an Abstract Syntax Tree (AST) because:

- It represents the **abstract structure** of the source code, focusing on the **essential meaning** rather than the exact textual details.
- Unlike a parse tree (concrete syntax tree), an **AST**:
 - **Omits unnecessary syntactic elements** such as parentheses, commas, or certain keywords that do not affect the semantics.

- **Simplifies hierarchical relationships** to show only the logical structure of the program.
- The term “**abstract**” refers to this simplification and abstraction from the full grammar-based representation.

ASTs are needed because of the inherent nature of programming languages and their documentation. Languages are often ambiguous by nature. In order to avoid this ambiguity, programming languages are often specified as a context-free grammar (CFG). However, there are often aspects of programming languages that a CFG can't express, but are part of the language and are documented in its specification. These are details that require a context to determine their validity and behaviour.

For example, if a language allows new types to be declared, a CFG cannot predict the names of such types nor the way in which they should be used. Even if a language has a predefined set of types, enforcing proper usage usually requires some context.

Another example is duck typing, where the type of an element can change depending on context. Operator overloading is yet another case where correct usage and final function are determined based on the context. Java provides an excellent example, where the '+' operator is both numerical addition and concatenation of strings.

Although there are other data structures involved in the inner workings of a compiler, the AST performs a unique function. During the first stage, the syntax analysis stage, a compiler produces a parse tree. This parse tree can be used to perform almost all functions of a compiler by means of syntax-directed translation. Although this method can lead to a more efficient compiler, it goes against the software engineering principles of writing and maintaining programs. Another advantage that the AST has over a parse tree is the size, particularly the smaller height of the AST and the smaller number of elements.

Design patterns

Due to the complexity of the requirements for an AST and the overall complexity of a compiler, it is beneficial to apply sound software development principles. One of these is to use proven design patterns to enhance modularity and ease of development.

Different operations don't necessarily have different types, so it is important to have a sound node class hierarchy. This is crucial in the creation and the modification of the AST as the compiler progresses.

Because the compiler traverses the tree several times to determine syntactic correctness, it is important to make traversing the tree a simple operation. The compiler executes a specific set of operations, depending on the type of each node, upon reaching it, so it often makes sense to use the visitor pattern.